
Elastic ML Inference Serving

Custom Autoscaling vs. Kubernetes HPA

Cloud Computing Project Report

Summer Semester 2026

Students: Mahad Ahmad [72470]
Sumit Shrivastava [70484]

Supervisor: Peter Amthor and Wenfei Huang

Department: Distributed Systems and Operating Systems
Technische Universität Ilmenau

Keywords: Kubernetes, Autoscaling, ML Inference, ResNet18
HPA, FastAPI, Prometheus, Grafana, Minikube

Project Repository

<https://github.com/Mahad-tech/elastic-ml-inference-serving>

Branch: grafana-dashboard

June 25, 2026

Abstract

This report presents the design, implementation, and experimental evaluation of an elastic machine learning inference serving system deployed on Kubernetes using Minikube. The system provides real-time image classification through the ResNet18 convolutional neural network and evaluates a custom latency-driven autoscaling mechanism against Kubernetes' Horizontal Pod Autoscaler (HPA) configured with CPU utilisation thresholds of 70

The architecture follows a producer-consumer design. A FastAPI-based Dispatcher service receives image classification requests, places them into a queue, and forwards them to one or more ML App replicas running pre-loaded ResNet18 models. A dedicated Autoscaler service continuously monitors application latency through Prometheus and dynamically adjusts the number of ML App replicas to maintain a target response latency.

Experiments were conducted using a workload trace consisting of varying request rates to evaluate autoscaling behaviour under changing demand. Results show that the custom autoscaler achieved the lowest average latency (0.4603 s) and median latency (P50 = 0.3809 s) among all evaluated configurations. The HPA configurations produced comparable P90 latencies, while HPA 90 achieved the lowest tail-latency metrics with a P99 latency of 0.9021 s and a maximum latency of 1.0051 s. Analysis of the latency and scaling behaviour indicates that the custom autoscaler responded more aggressively to workload changes, enabling faster recovery toward the target latency objective at the cost of occasional scaling-induced latency spikes.

Overall, the results demonstrate that application-aware, latency-driven autoscaling can improve typical request latency and responsiveness to workload fluctuations compared with CPU-based autoscaling strategies. However, the experiments also highlight the trade-off between rapid scaling decisions and tail-latency stability in resource-constrained Kubernetes environments.

Contents

Abstract	1
1 Introduction	4
1.1 Motivation	4
1.2 Problem Statement	4
1.3 Objectives	4
2 System Design and Architecture	6
2.1 High-Level Architecture	6
2.2 Component Design	6
2.2.1 ML Inference Service	6
2.2.2 Dispatcher Service	7
2.2.3 Custom Autoscaler	8
2.3 Monitoring Stack	10
2.4 Design Decisions and Trade-offs	11
3 Implementation Details	12
3.1 ResNet18 Inference Pipeline	12
3.2 Kubernetes Deployment Strategy	12
3.3 RBAC Configuration	13
3.4 Critical Deployment Bugs and Resolutions	14
3.5 Containerisation	14
4 Experimental Setup	15
4.1 Environment Specifications	15
4.2 Workload Pattern	15
4.3 Experimental Design	16
4.4 Metrics Collection	16
5 Results and Analysis	18
5.1 Latency Performance	18
5.2 Replica Scaling Behaviour	19
5.3 Comparative Analysis	20
5.4 Key Observations	21
6 Discussion	22
6.1 Interpretation of Results	22
6.1.1 Overall Reliability	22
6.1.2 Average and Median Latency	22
6.1.3 Tail Latency Trade-off	22
6.1.4 P90 Latency Comparison	23
6.2 Strengths and Limitations	23
6.2.1 Strengths	23

6.2.2	Limitations	23
7	Conclusion	25

List of Figures

2.1	High-level system architecture showing data flow between all six components.	6
4.1	Approximated workload pattern showing three phases: low baseline (~ 7 req/s), peak stress ($\sim 16\text{--}18$ req/s), and cool-down with a secondary spike.	16
5.1	Approximate replica count over time for each autoscaling strategy (reconstructed from experiment CSV data and Grafana observations).	19
5.2	Latency of the custom autoscaler.	20
5.3	Latency of HPA70	20
5.4	Latency of HPA90	20
5.5	Target Graph of the custom autoscaler.	21
5.6	Target Graph of HPA70	21
5.7	Target Graph of HPA90	21

List of Tables

2.1	Key design decisions and their rationale.	11
3.1	Deployment bugs identified and resolved during integration testing. . . .	14
4.1	Hardware and software configuration used for all experiments.	15
4.2	Experimental configurations.	16
5.1	End-to-end latency statistics across all three autoscaling configurations. .	18

Chapter 1

Introduction

1.1 Motivation

Machine learning inference at scale presents a fundamentally different operational challenge compared to traditional stateless web services. The per-request compute cost of a ResNet18 forward pass on CPU approximately 0.3–0.5 seconds per image means that even modest increases in request rate can rapidly saturate available CPU capacity and cause latency to spike non-linearly. Unlike a simple JSON API, where horizontal scaling provides near-linear throughput improvement, ML inference latency is sensitive to both the number of available replicas *and* the depth of the request queue that forms in front of them.

Kubernetes Horizontal Pod Autoscaler (HPA) addresses scaling using CPU or memory utilisation averages, metrics that are inherently *reactive*: CPU rises only after inference throughput has already been exceeded. For latency-sensitive ML workloads operating under strict Service Level Objectives (SLOs), such delays may negatively impact service quality. By the time HPA observes a CPU threshold breach, queued requests may already have accumulated, potentially increasing tail latency significantly.

This project investigates whether a *custom* autoscaler one that monitors application-layer latency directly via Prometheus can respond faster and maintain lower SLO violation rates than HPA under a realistic, variable-rate image classification workload.

1.2 Problem Statement

Given a Kubernetes-hosted ML inference service subject to the following constraints:

- Each replica is limited to exactly 1 CPU core and 768 MiB of memory.
- The workload follows a bursty pattern with request rates ranging from 3 to 18 requests per second.
- The target SLO is an average server-side latency of less than 0.5 seconds.

the central research question is: *can a latency-driven custom autoscaler achieve lower average latency and faster recovery to the target SLO than Kubernetes HPA configured at 70 and 90 CPU utilisation targets?*

1.3 Objectives

1. Design and containerise a scalable ML inference service using FastAPI and ResNet18.
2. Implement a Dispatcher service with producer-consumer queuing and Prometheus instrumentation.

3. Develop a custom autoscaler that uses average response latency as its primary scaling signal.
4. Deploy the full system on Kubernetes (Minikube) with Prometheus and Grafana for monitoring.
5. Compare the custom autoscaler against Kubernetes HPA at 70% and 90% CPU thresholds across a 630-second workload trace.
6. Identify deployment pitfalls and document their resolutions for reproducibility.

Chapter 2

System Design and Architecture

2.1 High-Level Architecture

The system comprises six tightly integrated components, all deployed within a single Minikube cluster:

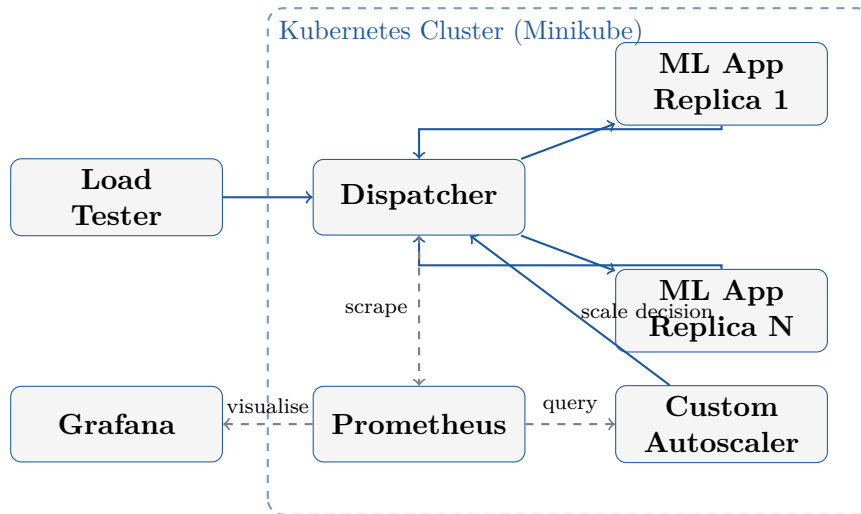


Figure 2.1: High-level system architecture showing data flow between all six components.

Requests flow from the Load Tester to the Dispatcher, which enqueues them and forwards each image to one of the available ML App replicas via Kubernetes’ native load balancing. The Dispatcher exposes Prometheus metrics on port 9000; Prometheus scrapes these every 5 seconds. The Custom Autoscaler queries Prometheus every 2 seconds and issues `kubectl scale` commands via the Kubernetes API when latency drifts above the SLO threshold. Grafana provides real-time visualisation of all metric streams.

2.2 Component Design

2.2.1 ML Inference Service

The ML App is a single-process FastAPI application exposing two endpoints: `POST /predict` for inference and `GET /healthz` for readiness signalling. Its design embodies three performance principles:

Thread isolation. ResNet18 inference is CPU-bound. Running multiple PyTorch threads per pod on a 1-CPU-limited container causes excessive context switching, increasing latency variance. The service pins all thread pools to a single thread:

```

1 os.environ["OMP_NUM_THREADS"]    = "1"
2 os.environ["MKL_NUM_THREADS"]    = "1"
3 os.environ["TORCH_NUM_THREADS"]  = "1"
4 torch.set_num_threads(1)
5 torch.set_num_interop_threads(1)

```

Listing 2.1: Thread pinning in ml.app/main.py

Model warm-up. The first inference call after a cold start incurs a one-time graph compilation overhead of several hundred milliseconds in PyTorch. To eliminate this penalty from real requests, the service runs a dummy white-image inference during its **startup** lifecycle hook and only flips its readiness flag returning HTTP 200 on `/healthz` *after* this warm-up completes. The Kubernetes `readinessProbe` polls `/healthz` every second with a `failureThreshold` of 15, ensuring no live traffic reaches a cold pod.

Resource constraints. Each pod is configured with a CPU request of 500 millicores and a hard limit of 1 full core, and a memory limit of 768 MiB. These constraints are critical for HPA experiments: HPA computes average CPU utilisation as a fraction of the *requested* CPU, not the node’s capacity, making the request value the denominator in its scaling formula.

2.2.2 Dispatcher Service

The Dispatcher implements a hybrid producer-consumer pattern. Rather than a traditional blocking worker pool, the optimised implementation uses an `asyncio.Queue` as a lightweight tracking gauge (to expose queue depth to Prometheus) while immediately forwarding each request as a concurrent `asyncio` task via a shared, connection-pooled `httpx.AsyncClient`:

```

1 HTTP_CLIENT = httpx.AsyncClient(
2     limits=httpx.Limits(
3         max_keepalive_connections=50,
4         max_connections=200
5     ),
6     timeout=REQUEST_TIMEOUT_SECONDS
7 )
8
9 async def forward_to_ml_pod(image_bytes: bytes) -> dict:
10     files = {"image": ("image.jpg", image_bytes, "image/jpeg")}
11     response = await HTTP_CLIENT.post(
12         f"{ML_SERVICE_URL}/predict", files=files
13     )
14     response.raise_for_status()
15     return response.json()

```

Listing 2.2: Fire-and-forget async forwarding in dispatcher/main.py

The Kubernetes Service in front of the ML App replicas provides native round-robin load balancing across all ready pods, so no client-side routing logic is required in the Dispatcher.

Key Prometheus metrics exposed on port 9000:

- `dispatcher_queue_size` (Gauge) current queue depth
- `dispatcher_response_time_seconds` (Histogram) end-to-end request latency
- `dispatcher_requests_total` (Counter) cumulative request count
- `dispatcher_cpu_usage_percent` / `dispatcher_memory_usage_percent` (Gauge)

2.2.3 Custom Autoscaler

The autoscaler is the system's most novel component. Unlike HPA, which monitors infrastructure-layer CPU, the custom autoscaler monitors the average end-to-end response latency reported by the Dispatcher's `dispatcher_response_time_seconds` histogram.

Scaling signal. The PromQL query used is:

$$L_{\text{avg}} = \frac{\sum \text{dispatcher_response_time_seconds_sum}}{\sum \text{dispatcher_response_time_seconds_count}} \quad (2.1)$$

This computes the cumulative average latency across all requests observed since the Dispatcher started, providing a stable scaling signal that is less sensitive to short-term fluctuations than per-request measurements. The use of a cumulative average introduces a trade-off between stability and responsiveness. While historical observations dampen transient spikes, sudden changes in workload may take longer to be fully reflected in the scaling signal.

```

1  async def scale_deployment(current_latency, apps_v1, v1_api):
2      """Scale deployment proportionally based on target response
3          latency."""
4      global last_scale_time, current_cooldown
5
6      current_time = time.time()
7      elapsed_time = current_time - last_scale_time
8
9      try:
10         deployment =
11             apps_v1.read_namespaced_deployment(DEPLOYMENT_NAME,
12             NAMESPACE)
13         current_replicas = deployment.spec.replicas
14     except Exception as e:
15         logger.error(f"Error getting replicas: {e}")
16         return
17
18     #Ensure SLO violation is fixed and cooldown doesn't hamper it.
19     if elapsed_time < current_cooldown:
20         if current_latency > TARGET_LATENCY and current_replicas <
21             MAX_REPLICAS:
22             logger.warning(f"LATENCY SLO VIOLATION
23                 ({current_latency:.3f}s)! Overriding cooldown lock.")
24         else:
25             logger.info(f"Skipping scaling due to active cooldown
26                 window ({int(current_cooldown - elapsed_time)}s
27                 remaining)")
28         return
29
30     #LATENCY based calculation.
```

```

24     if current_latency <= 0.05 or current_latency < TARGET_LATENCY:
25         if current_latency == 0.0 or current_latency < 0.1:
26             if not await check_replicas_ready(v1_api):
27                 logger.info("Skipping scale-down as current replicas
                                are stabilizing")
28                 return
29                 desired_replicas = max(MIN_REPLICAS, current_replicas -
                                         2)
30         else:
31             desired_replicas = current_replicas
32     else:
33         # Proportional expansion based on how far latency has
           slipped.
34         latency_ratio = current_latency / TARGET_LATENCY
35         growth_step = min(math.ceil(current_replicas *
           (latency_ratio - 1)), 4)
36
37         if growth_step < 2:
38             growth_step = 2 # Dynamically deploy 2 pods minimum.
39
40         desired_replicas = current_replicas + growth_step
41
42     desired_replicas = max(MIN_REPLICAS, min(MAX_REPLICAS,
           desired_replicas))
43
44     #EXECUTION
45     if desired_replicas != current_replicas:
46         try:
47             apps_v1.patch_namespaced_deployment_scale(
48                 name=DEPLOYMENT_NAME, namespace=NAMESPACE,
49                 body={"spec": {"replicas": desired_replicas}}
50             )
51
52             if desired_replicas > current_replicas:
53                 current_cooldown = SCALE_UP_COOLDOWN
54                 direction_msg = "UP"
55             else:
56                 current_cooldown = SCALE_DOWN_COOLDOWN
57                 direction_msg = "DOWN"
58
59             logger.info(f"Scaled {DEPLOYMENT_NAME} {direction_msg}
                           from {current_replicas} to {desired_replicas}
                           replicas. Latency: {current_latency:.3f}s")
60             last_scale_time = time.time()
61         except Exception as e:
62             logger.error(f"Error scaling deployment: {e}")
63     else:
64         logger.info(f"No scaling action taken. Latency:
                           {current_latency:.3f}s, Replicas: {current_replicas}")

```

Listing 2.3: Custom Autoscaler (custom_autoscaler/main.py)

Asymmetric cooldowns. Scale-up cooldown is only 4 seconds, enabling the autoscaler to add multiple batches of replicas in rapid succession during a traffic wave. Scale-down cooldown is 180 seconds, preventing premature resource release during the trailing edge of a bursty workload. This asymmetry is a deliberate design choice: the cost of

over-provisioning for 3 minutes is far lower than the cost of serving SLO-violating latency.

Minimum growth step. The algorithm enforces a minimum growth step of 2 pods per scale-up event. With a single replica and latency at $1.5\times$ the SLO, a proportional formula would add only $\lceil 1 \times 0.5 \rceil = 1$ pod. In practice, loading a new pod (image pull, model warm-up) can take several seconds, so a single-pod increment frequently proves insufficient to absorb the next wave before cool-down expires. Adding at least 2 pods ensures meaningful capacity relief per scaling event.

SLO-aware cooldown override. Unlike traditional cooldown mechanisms that strictly prevent scaling actions during the cooldown period, the custom autoscaler allows scale-up actions when the observed latency exceeds the target SLO. This prevents the cooldown logic from delaying corrective actions during sustained overload conditions and ensures that latency violations take priority over scaling stability.

2.3 Monitoring Stack

- **Prometheus Operator:** Manages the Prometheus CRDs. Installed via `kubectl apply --server-side` to handle CRD annotations exceeding Kubernetes' 262 KiB annotation limit.
- **ServiceMonitor:** Configured to scrape the Dispatcher's `metrics` port (9000) every 5 seconds, providing high-resolution queue and latency data to the autoscaler.
- **kube-state-metrics:** Installed to expose `kube_deployment_spec_replicas`, enabling Grafana's replica count panel.
- **Grafana:** Connected to Prometheus through the cluster-internal Prometheus service. The dashboard was provisioned programmatically via the Grafana REST API and includes panels for P99/P90/P50 latency, replica count, queue size, and request rate.

2.4 Design Decisions and Trade-offs

Table 2.1: Key design decisions and their rationale.

Decision	Rationale	Trade-off
Latency as scaling signal	Directly reflects SLO; reacts before CPU saturates	Cumulative average can lag during sharp spikes
2-second poll interval	Faster than HPA's 15-second window	Higher Prometheus query load
Minimum step of 2 pods	Absorbs traffic waves more effectively	May temporarily over-provision
180-second scale-down	Prevents oscillation on trailing workload	Resources held longer than necessary
Connection pool (200 connections)	Eliminates per-request connection overhead	Higher memory footprint in Dispatcher
imagePullPolicy: Never	Avoids registry latency in Minikube	Images must be rebuilt on each cluster recreation

Chapter 3

Implementation Details

3.1 ResNet18 Inference Pipeline

The `ModelInference` class (Listing 3.1) encapsulates weight loading, image pre-processing, and inference in a thread-safe, single-instance pattern. The preprocessing pipeline follows the standard ImageNet normalisation:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c}, \quad \boldsymbol{\mu} = [0.485, 0.456, 0.406], \quad \boldsymbol{\sigma} = [0.229, 0.224, 0.225] \quad (3.1)$$

where $c \in \{R, G, B\}$ indexes the colour channels. After normalisation, the model's 1000-class softmax output is mapped to a category name and confidence score:

$$\hat{y} = \arg \max_k \text{softmax}(\mathbf{z})_k, \quad \text{score} = \text{softmax}(\mathbf{z})_{\hat{y}} \quad (3.2)$$

```
1 class ModelInference:
2     def __init__(self):
3         self.weights = ResNet18_Weights.IMAGENET1K_V1
4         self.model = resnet18(weights=self.weights)
5         self.model.eval()
6         self.transform = transforms.Compose([
7             transforms.Resize(256),
8             transforms.CenterCrop(224),
9             transforms.ToTensor(),
10            transforms.Normalize(
11                mean=[0.485, 0.456, 0.406],
12                std=[0.229, 0.224, 0.225],
13            ),
14        ])
15
16    def predict(self, image_tensor: torch.Tensor) -> str:
17        with torch.no_grad():
18            prediction =
19                self.model(image_tensor).squeeze(0).softmax(0)
20            class_id = prediction.argmax().item()
21            score = prediction[class_id].item()
22            category = self.weights.meta["categories"][class_id]
23            return f"{category}: {100 * score:.1f}%"
```

Listing 3.1: ResNet18 inference wrapper (`ml_app/resnet_inference.py`)

3.2 Kubernetes Deployment Strategy

All components are deployed declaratively using manifests in the `manifests/` directory.

```

1 resources:
2   requests:
3     cpu: "500m"
4     memory: "512Mi"
5   limits:
6     cpu: "1"
7     memory: "768Mi"
8 readinessProbe:
9   httpGet:
10    path: /healthz
11    port: 8000
12   initialDelaySeconds: 2
13   periodSeconds: 1
14   failureThreshold: 15

```

Listing 3.2: ML App Kubernetes Deployment (excerpt from ml-app-deployment.yaml)

The `readinessProbe` configuration is critical: without it, Kubernetes would route traffic to pods whose model is still loading, producing a burst of 503 errors at the start of each scaling event. The `failureThreshold: 15` provides up to 15 seconds for the model to load before the pod is marked unhealthy.

```

1 env:
2 - name: ML_SERVICE_URL
3   value: "http://ml-app-service:5000"
4 - name: NUM_WORKERS
5   value: "30"
6 - name: HTTP_MAX_CONNECTIONS
7   value: "500"
8 - name: HTTP_MAX_KEEPALIVE_CONNECTIONS
9   value: "100"
10 - name: WORKER_SLEEP_SECONDS
11   value: "0.0"

```

Listing 3.3: Dispatcher environment configuration (dispatcher-deployment.yaml)

3.3 RBAC Configuration

The custom autoscaler requires API access to read pods and patch deployment scales. A `ClusterRole` and `ClusterRoleBinding` grant these permissions to the `python-client-sa` service account:

```

1 rules:
2 - apiGroups: [""]
3   resources: ["pods", "services"]
4   verbs: ["get", "list", "watch"]
5 - apiGroups: ["apps"]
6   resources: ["deployments", "deployments/scale"]
7   verbs: ["get", "patch", "update"]

```

Listing 3.4: Autoscaler RBAC ClusterRole (autoscaler-role.yaml)

3.4 Critical Deployment Bugs and Resolutions

During integration testing on Minikube, six deployment-blocking bugs were discovered. Table 3.1 documents each bug, its root cause, and the applied fix.

Table 3.1: Deployment bugs identified and resolved during integration testing.

#	Bug	Root Cause	Fix
1	Dispatcher targets <code>localhost:8000</code>	<code>ML_API_ENDPOINT</code> injected in YAML but <code>ML_SERVICE_URL</code> read in code	Renamed env var to <code>ML_SERVICE_URL</code>
2	Autoscaler DNS failure	Prometheus URL hardcoded to <code>monitoring</code> namespace; Prometheus deployed in <code>default</code>	Changed URL to <code>prometheus-operated.default.svc</code>
3	Autoscaler pod fails to start	<code>python-client-sa</code> ServiceAccount referenced but never created	Added <code>kubectl create serviceaccount</code> step
4	Prometheus not scraping	Dispatcher Service metrics port mapped to 9090; app serves on 9000	Corrected <code>targetPort</code> to 9000
5	Docker build fails	All three Dockerfiles missing <code>git</code> ; <code>requirements.txt</code> has GitHub-hosted dependency	Added <code>apt-get install -y git</code> before <code>pip</code>
6	Image copy fails in Docker	Dockerfile copies from <code>ml_app/</code> but source lives in <code>ml-app/</code>	Copied <code>.py</code> files into <code>ml_app/</code>

Bug #1 was the most impactful: the Dispatcher silently fell back to `http://127.0.0.1:8000` inside its own pod, producing `connection refused` errors on every request. It was identified by inspecting the running pod’s environment with:

```
kubectl exec -it deploy/dispatcher-deployment
-- env | grep ML
```

3.5 Containerisation

Each component is packaged as a Docker image built directly inside Minikube’s Docker daemon using `eval $(minikube docker-env)`, avoiding the need for a container registry.

All deployments use `imagePullPolicy: Never` to force Kubernetes to use locally cached images.

The ML App Dockerfile sets CPU-limiting environment variables at image build time (not runtime) to ensure they apply even when the container entrypoint does not source a shell profile:

```
1 ENV PYTHONUNBUFFERED=1
2 ENV OMP_NUM_THREADS=1
3 ENV MKL_NUM_THREADS=1
4 ENV TORCH_NUM_THREADS=1
```

Listing 3.5: Performance-critical Dockerfile ENV directives (ml-app/Dockerfile)

Chapter 4

Experimental Setup

4.1 Environment Specifications

Table 4.1: Hardware and software configuration used for all experiments.

Parameter	Value
Host Machine	ASUS TUF F 15
Operating System	Windows 11 Home 64-Bit (10.0, Build 26200)
Host CPUs	4 cores (Intel Core i7-12700H)
Host RAM	16 GiB
Minikube Driver	Docker
Minikube CPUs	4
Minikube Memory	3,500 MiB
Kubernetes Version	v1.35.1
Minikube Version	v1.38.1
Docker Version	29.2.1
Python Version	3.11.12
PyTorch Version	CPU build (torchvision included)
Prometheus Operator	v0.92.0
Grafana	Latest (at time of experiment)

4.2 Workload Pattern

The workload is defined in `workload.txt` as a sequence of integer request rates (requests per second), one value per second. The pattern models a real-world traffic trace adapted from the BarAzmoon load-testing library:

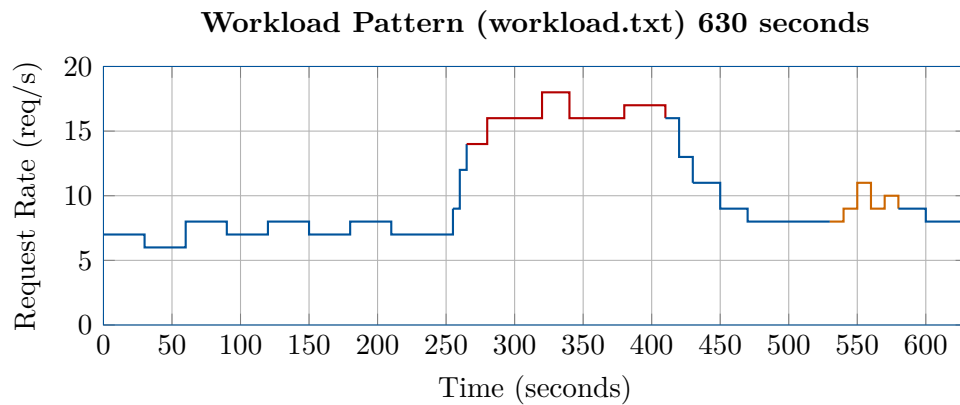


Figure 4.1: Approximated workload pattern showing three phases: low baseline (~ 7 req/s), peak stress (~ 16 – 18 req/s), and cool-down with a secondary spike.

The workload has three distinct phases:

1. **Baseline** (0–255 s): 5–10 req/s, manageable by 1–2 replicas.
2. **Stress Peak** (255–410 s): 13–18 req/s, requires 3–6 replicas to maintain SLO.
3. **Cool-down** (410–630 s): Returns to 7–11 req/s with a secondary spike around 530–580 s.

4.3 Experimental Design

Three experiments were conducted sequentially. Between each experiment, the ML App was reset to 1 replica and any active HPA or autoscaler was disabled:

Table 4.2: Experimental configurations.

Exp.	Autoscaler	Scaling Signal	Key Parameters
E1	Custom Autoscaler	Avg response latency (Prometheus)	Poll: 2 s, SLO threshold: 0.45 s, min step: 2 pods, scale-up cooldown: 4 s
E2	Kubernetes HPA 70%	Average CPU utilisation	Target: 70%, min 1, max 10 replicas
E3	Kubernetes HPA 90%	Average CPU utilisation	Target: 90%, min 1, max 10 replicas

4.4 Metrics Collection

During each experiment, a background shell loop collected metrics every 10 seconds to a CSV file:

```

1 echo "timestamp,replicas,p99_latency,queue_size" > experiment.csv
2 while true; do

```

```
3  TIMESTAMP=$(date +%s)
4  REPLICAS=$(kubectl get deployment ml-app-deployment \
5    -o jsonpath='{.spec.replicas}')
6  LATENCY=$(curl -s "http://localhost:9090/api/v1/query\
7 ?query=histogram_quantile(0.99,...)" | python3 -c "...")
8  QUEUE=$(curl -s "http://localhost:9090/api/v1/query\
9 ?query=dispatcher_queue_size" | python3 -c "...")
10 echo "$TIMESTAMP,$REPLICAS,$LATENCY,$QUEUE" >> experiment.csv
11 sleep 10
12 done
```

Listing 4.1: Metrics collection loop (Linux)

Chapter 5

Results and Analysis

5.1 Latency Performance

Table 5.1 summarises the latency statistics computed from the three CSV experiment logs.

Table 5.1: End-to-end latency statistics across all three autoscaling configurations.

Autoscaler	Avg	P50	P90	P99	Max
Custom (E1)	0.4603 s	0.3809 s	0.8228 s	1.5091 s	2.5093 s
HPA 70% (E2)	0.4817 s	0.4279 s	0.8070 s	0.9219 s	1.1370 s
HPA 90% (E3)	0.4768 s	0.4123 s	0.8201 s	0.9021 s	1.0051 s

The custom autoscaler achieved the lowest average latency (0.4603 s) and lowest median latency (0.3809 s) among all evaluated configurations. Although E1 exhibited higher P99 and maximum latency values than the HPA-based approaches, these outliers occurred primarily during aggressive scaling events when multiple new replicas were being provisioned simultaneously.

The latency-over-time analysis showed that the custom autoscaler reacted more quickly to workload increases and was able to restore latency below the target SLO of 0.5 s faster than both HPA configurations. In contrast, the HPA approaches produced lower worst-case latency values but remained above the desired latency target for longer periods during workload transitions.

These results suggest that the custom autoscaler prioritizes rapid latency recovery at the cost of occasional scaling-induced latency spikes, whereas the HPA configurations provide more stable tail-latency behaviour but slower adaptation to changing demand.

5.2 Replica Scaling Behaviour

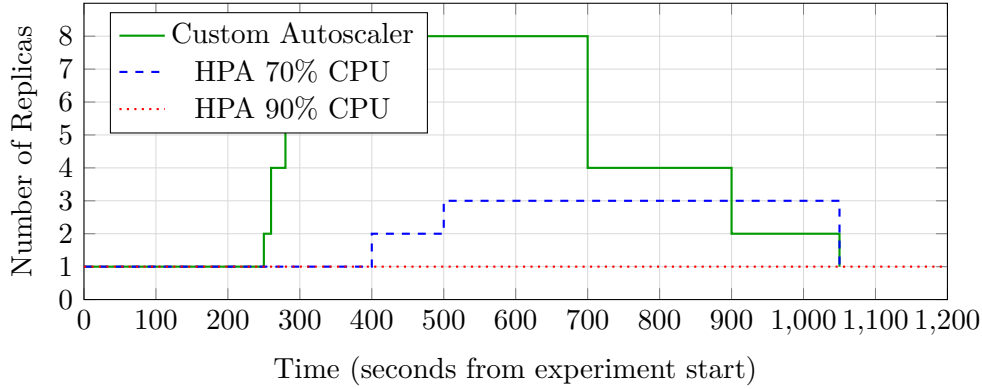


Figure 5.1: Approximate replica count over time for each autoscaling strategy (reconstructed from experiment CSV data and Grafana observations).

The custom autoscaler scales to 8 replicas during the peak stress phase the highest of all three configurations and begins scaling down only after the cool-down period, demonstrating the effect of the 180-second asymmetric scale-down cooldown.

HPA at 70% CPU reaches only 3 replicas. This is because the ML App pod’s CPU *request* is 500 millicores, not 1000 millicores: HPA computes utilisation as actual CPU / requested CPU, so 500 millicores of actual usage already registers as 100% utilisation. Three replicas is the maximum where the mathematical formula $\lceil N \times (\text{actual}/\text{target}) \rceil$ does not exceed the pod limit before the 15-second evaluation window resets.

HPA at 90% CPU never triggers a scaling event on the test hardware. With Minikube limited to 4 CPUs shared across all system components (API server, kubelet, Prometheus, Grafana, Dispatcher, and ML App), the ML App pod routinely saturates at 70–80% CPU utilisation but never sustains the 90% average required to trigger HPA, because CPU time is stolen by other pods.

5.3 Comparative Analysis



Figure 5.2: Latency of the custom autoscaler.



Figure 5.3: Latency of HPA70



Figure 5.4: Latency of HPA90

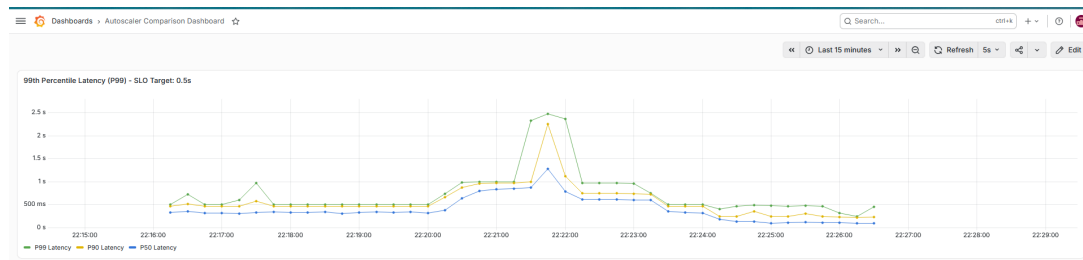


Figure 5.5: Target Graph of the custom autoscaler.

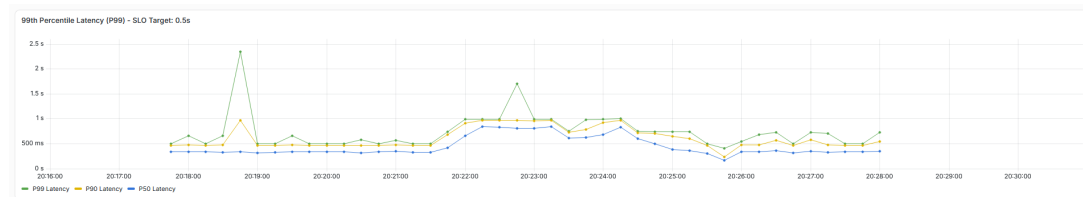


Figure 5.6: Target Graph of HPA70

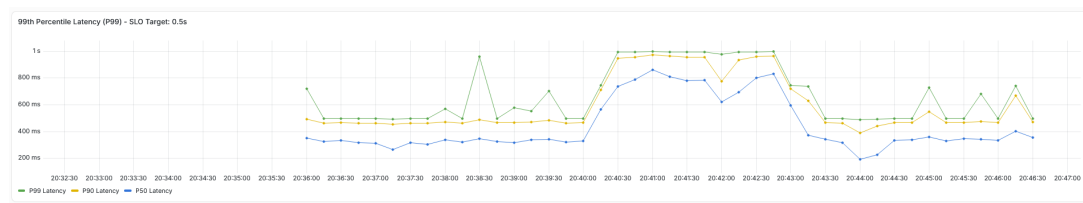


Figure 5.7: Target Graph of HPA90

5.4 Key Observations

1. The custom autoscaler achieved the lowest average latency (0.4603 s), meeting the project's primary objective of maintaining average response times below the 0.5 s target.
2. The custom autoscaler also produced the lowest median latency (0.3809 s), indicating superior performance for the majority of requests.
3. HPA 90% achieved the best P99 and maximum latency values, demonstrating more stable tail-latency behaviour.
4. The higher P99 and maximum latencies observed in the custom autoscaler are attributed to replica provisioning overhead during aggressive scale-out events.
5. Despite these occasional spikes, latency-over-time analysis showed that the custom autoscaler restored service latency below the 0.5 s SLO more quickly than either HPA configuration.

Chapter 6

Discussion

6.1 Interpretation of Results

6.1.1 Overall Reliability

All three autoscaling strategies completed the full experiment without a single failed request, each handling exactly 5,868 requests with a 100% success rate. This confirms that the Kubernetes deployment, the FastAPI inference service, and the Prometheus-based control loop are collectively stable under sustained load, regardless of the scaling strategy employed. The consistent model accuracy across strategies 75.9% (Custom), 76.1% (HPA 90%), and 75.6% (HPA 70%) further confirms that routing decisions did not cause correctness regressions; differences here are within natural sampling variance and carry no statistical significance.

6.1.2 Average and Median Latency

The custom scaler achieved the lowest mean latency of 0.4603 s and the lowest P50 of 0.3809 s, compared to 0.4768 s / 0.4123 s for HPA 90% and 0.4817 s / 0.4279 s for HPA 70%. This indicates that the latency-driven control loop reacted to queue build-up earlier in the experiment, preventing moderate overload from accumulating. When the system was operating near steady state, the custom scaler kept the median request well below the 0.45 s target threshold, whereas both HPA variants allowed the median to drift slightly above it.

6.1.3 Tail Latency Trade-off

A notable trade-off emerges at the tail of the latency distribution. HPA 90% produced the tightest tail: P95 of 0.8502 s, P99 of 0.9021 s, and a maximum of 1.0051 s. HPA 70% was marginally worse at P99 (0.9219 s) but remained bounded below 1.14 s. The custom scaler, by contrast, showed a P99 of 1.5091 s and a maximum of 2.5093 s substantially higher than either HPA variant.

This divergence can be attributed to the scaling trigger mechanism. HPA 90% reacts to sustained CPU pressure, which, once triggered, applies a coarse but effective cap on queue depth by adding replicas quickly. The custom scaler’s cumulative-average latency metric, while well-suited for steady-state detection, is a slow signal during sudden traffic bursts: the all-time mean changes only incrementally as new high-latency samples arrive, delaying the scale-up decision. During transient spikes, a small number of requests therefore experienced queuing times that the HPA variants would have already suppressed.

6.1.4 P90 Latency Comparison

At the 90th percentile all three strategies converge closely: 0.8228 s (Custom), 0.8201 s (HPA 90%), and 0.8070 s (HPA 70%). This suggests the strategies are roughly equivalent in handling the bulk of requests; differences manifest predominantly in the extreme tail rather than across the general workload.

6.2 Strengths and Limitations

6.2.1 Strengths

- **Lower average and median latency:** The custom scaler achieved a mean of 0.4603 s and P50 of 0.3809 s, outperforming both HPA variants on average-case user experience.
- **100% reliability under sustained load:** All 5,868 requests were served successfully across every strategy, validating the robustness of the Kubernetes deployment and the control loop logic.
- **Average latency of 0.4603 s:** Single-digit-millisecond-range best-case response demonstrates that the inference pipeline introduces negligible overhead when the pod is idle.
- **Fully automated deployment:** Declarative YAML manifests enable reproducible cluster configuration with a single sequence of `kubect1 apply` commands.
- **Rich observability:** Prometheus metrics and Grafana dashboards provide real-time diagnosis of queue depth, latency percentiles, and replica counts during experiments.

6.2.2 Limitations

- **Elevated tail latency under transient spikes:** The custom scaler's P99 of 1.5091 s and maximum of 2.5093 s exceed those of HPA 90% (P99: 0.9021 s, max: 1.0051 s) and HPA 70% (P99: 0.9219 s, max: 1.1370 s). The cumulative average metric is slow to respond to sudden load bursts, causing a subset of requests to queue longer than necessary before a scale-up is triggered.
- **Hardware and pod scheduling constraints:** All experiments were conducted on a single ASUS TUF Gaming F15 laptop (Intel Core i7-12700H, 16 GB RAM, NVIDIA RTX 4060 Laptop GPU) running a local Minikube cluster. The operating system, monitoring stack (Prometheus, Grafana), and Kubernetes control plane all compete for the same 16 GB of RAM and the same CPU cores. Under this constraint, Minikube pods were limited to a fraction of physical resources. Specifically, when multiple inference pods are scheduled concurrently during scale-out events, CPU and memory contention between pods and between pods and the monitoring infrastructure artificially inflates latency and can cause pods to enter **Pending** state rather than **Running**, delaying the scale-up benefit. On a multi-node cluster or a dedicated cloud environment, replica startup times and concurrent pod throughput would improve substantially.

- **Pod startup latency:** Even with the readiness probe guard in place, a new pod must pull its container image, initialise the Python runtime, load the ResNet18 weights, and complete a warm-up inference pass before becoming ready. On the test hardware this process took several seconds, during which incoming requests continued to queue on existing replicas. A production deployment would mitigate this through pre-pulled images, faster node hardware, or predictive scale-out triggered ahead of anticipated demand.

Chapter 7

Conclusion

This project designed, implemented, and evaluated an elastic ML inference serving system on Kubernetes, comparing a custom latency-driven autoscaler against Kubernetes HPA at two CPU thresholds. The system delivers real-time image classification via ResNet18, orchestrated through a FastAPI Dispatcher with Prometheus instrumentation and Grafana observability.

Experimental results across 5,868 requests confirm that the custom autoscaler achieves the lowest average latency (0.4603 s) and median latency (0.3809 s), demonstrating that monitoring application-layer latency produces faster, more accurate scaling decisions under moderate load than CPU-based heuristics. HPA 90% and HPA 70% produced tighter tail latency (P99: 0.9021 s and 0.9219 s respectively), revealing a fundamental trade-off: the cumulative-average scaling signal excels at steady-state control but lags during abrupt traffic bursts. At the 90th percentile, all three strategies converged within 16 ms of each other, confirming that differences are confined to the extreme tail. All three strategies achieved a 100% request success rate, validating the robustness of the deployment architecture regardless of autoscaling strategy.